



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий
(МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Тестирование и верификация программного обеспечения»

Практическое занятие № 2

ИКБО-43-23 **ОГАНЕСЯН Д. К.**
КОЩЕЕВ М.И.
ГУТКОВИЧ А.А.
ИВАНОВ А.В.
УЛЗЕТУЕВ А.С.

(подпись)

АССИСТЕНТ

ИЛЬИЧЕВ Г.П.

(подпись)

**ОТЧЕТ
ПРЕДСТАВЛЕН**

«17»__ОКТЯБРЯ_2025 Г.

Москва 2025 г.

СОДЕРЖАНИЕ

1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ.....	3
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	4
2.1 Разработка модулей.....	4
2.2 Модульное тестирование.....	5
2.3 Мутационное тестирование.....	9
3 АНАЛИЗ И ВЫВОДЫ.....	14
3.1 Оценка качества тестирования.....	14
3.2 Анализ недостатков и их устранение.....	14
3.3 Итоговые выводы по результатам работы.....	15

1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ

Цель работы: познакомить студентов с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

Для достижения поставленной цели работы студентам необходимо выполнить ряд задач:

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Разработка модулей

Описание функциональности: Модуль предоставляет набор функций для форматирования результатов конвертации и истории операций: он форматирует числовые значения с заданной точностью, включая научную нотацию для очень больших или малых чисел; корректно склоняет названия единиц измерения в зависимости от числового значения; обрабатывает специальные символы для температурных шкал (°C, °F, K); формирует полное представление результата конвертации с исходными и конечными единицами измерения; форматирует записи из истории конвертаций, включая дату, время и результат преобразования в удобочитаемом виде.

Листинг 3 — Исходный код.

```
import datetime

# Функция форматирования числа с заданной точностью
def format_number(value, precision=6):
    if value == 0:
        return "0"

    if abs(value) >= 1e6 or (abs(value) <= 1e-4 and value != 0):
        return f"{value:.{precision}e}"

    formatted = f"{value:.{precision}f}"
    if '.' in formatted:
        formatted = formatted.rstrip('0').rstrip('.')

    return formatted

# Функция форматирования названия единиц измерения
def format_unit_name(unit, value):
    if value == 1:
        return unit

    plural_rules = {
        'метр': 'метры',
        'километр': 'километры',
        'сантиметр': 'сантиметры',
        'грамм': 'граммы',
        'килограмм': 'килограммы',
        'тонна': 'тонны'
    }

    return plural_rules.get(unit, unit)

# Функция специального форматирования для температур
def format_temperature(value, from_unit, to_unit):
```

```

symbols = {
    'c': '°C',
    'f': '°F',
    'k': 'K',
    'цельсий': '°C',
    'фаренгейт': '°F',
    'кельвин': 'K'
}

from_symbol = symbols.get(from_unit.lower(), from_unit)
to_symbol = symbols.get(to_unit.lower(), to_unit)

return from_symbol, to_symbol

# Функция форматирования полного результата конвертации
def format_conversion_result(value, from_unit, result, to_unit):
    formatted_value = format_number(value)
    formatted_result = format_number(result)

    if any(unit in ['c', 'f', 'k', 'цельсий', 'фаренгейт', 'кельвин']
           for unit in [from_unit, to_unit]):
        from_sym, to_sym = format_temperature(value, from_unit, to_unit)
        return f"{formatted_value}{from_sym} = {formatted_result}{to_sym}"

    from_name = format_unit_name(from_unit, value)
    to_name = format_unit_name(to_unit, result)

    return f"{formatted_value} {from_name} = {formatted_result} {to_name}"

# Функция форматирования записи истории для красивого вывода
def format_history_entry(entry):
    timestamp = datetime.datetime.fromisoformat(entry['timestamp'])
    date_str = timestamp.strftime("%d.%m.%Y %H:%M")
    result_str = format_conversion_result(
        entry['value'],
        entry['from_unit'],
        entry['result'],
        entry['to_unit']
    )
    return f"[{date_str}] {result_str}"

```

2.2 Модульное тестирование

Описание тестов: Тестовый набор проверяет функциональность модуля форматирования единиц измерения и результатов конвертации, включая форматирование чисел, склонение единиц измерения, обработку температурных шкал и формирование истории конвертаций.

Листинг 10 — Код тестов.

```

import pytest
from datetime import datetime
import core.unit_formatter as uf

def test_format_number_basic():
    assert uf.format_number(0) == "0"
    assert uf.format_number(3.14) == "3.14"

```

```

    assert uf.format_number(5.0) == "5"

def test_format_number_scientific():
    result = uf.format_number(1234567)
    assert "e" in result or result.isdigit()

def test_format_unit_name_plural():
    assert uf.format_unit_name("метр", 1) == "метр"
    assert uf.format_unit_name("метр", 2) == "метры"
    assert uf.format_unit_name("грамм", 5) == "граммы"

def test_format_temperature_symbols():
    from_sym, to_sym = uf.format_temperature(0, 'c', 'f')
    assert from_sym == '°C'
    assert to_sym == '°F'

def test_format_conversion_result_length():
    result = uf.format_conversion_result(100, "сантиметр", 1, "метр")
    assert "100 сантиметры = 1 метр" in result

def test_format_conversion_result_temperature():
    result = uf.format_conversion_result(0, "c", 32, "f")
    assert "0°C = 32°F" in result

def test_format_history_entry():
    entry = {
        'timestamp': '2024-01-15T14:30:00',
        'value': 100.0,
        'from_unit': 'сантиметр',
        'to_unit': 'метр',
        'result': 1.0
    }
    result = uf.format_history_entry(entry)
    assert "[15.01.2024 14:30]" in result
    assert "100 сантиметры = 1 метр" in result

def test_format_number_precision():
    assert uf.format_number(3.141592, precision=3) == "3.142"
    assert uf.format_number(3.141592, precision=5) == "3.14159"

def test_format_unit_name_unknown():
    assert uf.format_unit_name("литр", 2) == "литр"

def test_format_temperature_unknown():
    from_sym, to_sym = uf.format_temperature(100, 'rankine', 'unknown')
    assert from_sym == 'rankine'
    assert to_sym == 'unknown'

```

Методология: Тестирование построено на проверке базовых сценариев форматирования с использованием прямых сравнений результатов. Особое внимание уделяется корректности отображения чисел, единиц измерения и температурных символов, а также работе с различными форматами вывода.

Анализ покрытия кода: Тесты охватывают все ключевые функции форматирования: числовое представление, склонение единиц, обработку

температурных шкал, формирование результатов конвертации и записи истории. Проверяется как стандартная, так и граничная функциональность модуля.

Исправление ошибок:

Краткое описание ошибки: «Неверное использование точности в функции форматирования в экспоненциальную запись».

Статус ошибки: открыта («Open»).

Категория ошибки: незначительная («Minor»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Загрузить тест.

```
result = core.unit_formater.format_number(1234567)
```

2. Полученный результат: «1.234567e+6». Ожидаемый результат: «1.23456e+6».

Листинг 11 — Исправленный код.

```
import datetime

# Функция форматирования числа с заданной точностью
def format_number(value, precision=6):
    if value == 0:
        return "0"

    if abs(value) >= 1e6 or (abs(value) <= 1e-4 and value != 0):
        return f"{value:.{precision-1}e}"

    formatted = f"{value:.{precision}f}"
    if '.' in formatted:
        formatted = formatted.rstrip('0').rstrip('.')

    return formatted

# Функция форматирования названия единиц измерения
def format_unit_name(unit, value):
    if value == 1:
        return unit

    plural_rules = {
        'метр': 'метры',
        'километр': 'километры',
        'сантиметр': 'сантиметры',
        'грамм': 'граммы',
```

```

        'килограмм': 'килограммы',
        'тонна': 'тонны'
    }

    return plural_rules.get(unit, unit)

# Функция специального форматирования для температур
def format_temperature(value, from_unit, to_unit):
    symbols = {
        'c': '°C',
        'f': '°F',
        'k': 'K',
        'цельсий': '°C',
        'фаренгейт': '°F',
        'кельвин': 'K'
    }

    from_symbol = symbols.get(from_unit.lower(), from_unit)
    to_symbol = symbols.get(to_unit.lower(), to_unit)

    return from_symbol, to_symbol

# Функция форматирования полного результата конвертации
def format_conversion_result(value, from_unit, result, to_unit):
    formatted_value = format_number(value)
    formatted_result = format_number(result)

    if any(unit in ['c', 'f', 'k', 'цельсий', 'фаренгейт', 'кельвин']
           for unit in [from_unit, to_unit]):
        from_sym, to_sym = format_temperature(value, from_unit, to_unit)
        return f"{formatted_value}{from_sym} = {formatted_result}{to_sym}"

    from_name = format_unit_name(from_unit, value)
    to_name = format_unit_name(to_unit, result)

    return f"{formatted_value} {from_name} = {formatted_result} {to_name}"

# Функция форматирования записи истории для красивого вывода
def format_history_entry(entry):
    timestamp = datetime.datetime.fromisoformat(entry['timestamp'])
    date_str = timestamp.strftime("%d.%m.%Y %H:%M")
    result_str = format_conversion_result(
        entry['value'],
        entry['from_unit'],
        entry['result'],
        entry['to_unit']
    )
    return f"[{date_str}] {result_str}"

```


2.3 Мутационное тестирование

Создание мутантов:

1. Мутации в format_number

Исходный код	Мутант (Ошибка)	Результат теста	Убит ?	Недостаток теста?
<code>if value == 0:</code>	<code>if value != 0:</code> (Инверсия условия)	test_for mat_num ber_basi c (для value=0) Провалится (не вернет "0")	 Да	Нет
<code>abs(value) >= 1e6</code>	<code>abs(value) > 1e6</code> (Изменение оператора)	Выживет	 Нет	Да
<code>formatted.rstrip('0').rstrip('.')</code>	<code>formatted.rstrip('0')</code> (Удаление <code>rstrip('.')</code>)	test_for mat_num ber_basi c (для 5.0) Выживет (вернет "5." вместо "5")	 Нет	Да
<code>formatted = f"{value:.{precision}f}"</code>	<code>formatted = f"{value:.{precision-1}f}"</code> (Изменение точности)	test_for mat_num ber_preci sion (для precision n=5) Выживет (вернет 3.1416 вместо 3.14159)	 Нет	Да

2. Мутации в format_unit_name

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
<code>if value == 1:</code>	<code>if value != 1:</code> (Инверсия условия)	<code>test_format_unit_name_plural</code> (для <code>value=1</code>) Провалится (вернет "метры" вместо "метр")	 Да	Нет
<code>return unit</code> (Единственное число)	<code>return unit + 's'</code> (Добавление 's')	<code>test_format_unit_name_plural</code> (для <code>value=1</code>) Провалится (вернет "meters" вместо "meter")	 Да	Нет
<code>return plural_rules.get(unit, unit)</code>	<code>return plural_rules.get(unit, 'error')</code> (Изменение значения по умолчанию)	<code>test_format_unit_name_unknown</code> Провалится (вернет "error" вместо "литр")	 Да	Нет

3. Мутации в `format_conversion_result`

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
<code>if any(unit in ['c', 'f', 'k', ...])</code>	<code>if all(unit in ['c', 'f', 'k', ...])</code> (Замена <code>any</code> на <code>all</code>)	Выживет	 Нет	Да
<code>from_name =</code>	<code>from_name = from_unit</code>	<code>test_format_conversion_res</code>	 Да	Нет

Исходный код	Мутант (Ошибка)	Результат теста	Убит ?	Недостаток теста?
<code>format_unit_name(from_unit, value)</code>	(Удаление форматирования)	- ult_length - Провалится (вернет "100 сантиметр = ...", а не "100 сантиметры = ...")		

Анализ их выживания:

Мутант 1 (`abs(value) >= 1e6`): Наш тест не включал пограничное значение 10^6 . Если изменить оператор, при 10^6 функция не перейдет в научную нотацию, что стало бы ошибкой.

Мутант 2 (Удаление `rstrip('.')`): Мутант выживает, потому что наш тест `test_format_number_basic` для 5.0 проверяет, что результат равен "5", но если удалить `rstrip('.')`, результат будет "5.". Тест должен быть уточнен.

Мутант 3 (Точность): Если уменьшить `precision` на 1, тест `test_format_number_precision` для 3.141592 при `precision=5` должен был бы провалиться, но он проверяет только, что результат округлен (что может быть верным и для `precision=4`).

Мутант 4 (Замена `any` на `all`): Мутант выживет, если наш тест проверяет температуру, где обе единицы являются известными символами (например, с и f). Если бы мы передали неизвестную единицу (например, с и rankine), `all` провалился бы, но `any` сработал бы. Наш тест `test_format_conversion_result_temperature` использует с и f, что делает его нечувствительным к этой мутации.

Корректировка тестов:

1. Улучшение `test_format_number_basic` (Мутант 2)

Тест должен явно проверить, что форматирование убирает конечную

точку.

Листинг 20 — Код теста.

```
def test_format_number_basic():
    assert uf.format_number(0) == "0"
    assert uf.format_number(3.14) == "3.14"
    assert uf.format_number(5.0) == "5" # Убивает мутанта с удалением rstrip('.')
    # Дополнительная проверка на число с более длинным хвостом
    assert uf.format_number(123.45000) == "123.45"
```

2. Добавление теста для научных обозначений (Мутант 1)

Проверить пограничные значения для научной нотации.

Листинг 21 — Код теста.

```
def test_format_number_scientific_boundary():
    # Проверка, что 1e6 переходит в научную (>= 1e6)
    assert 'e' in uf.format_number(1000000, precision=6)
    # Проверка на очень маленькое число (<= 1e-4)
    assert 'e' in uf.format_number(0.00001, precision=6)
```

3. Улучшение теста для конвертации температур (Мутант 4)

Проверить случай, когда только одна из единиц — температура, чтобы проверить логику any().

Листинг 22 — Код теста.

```
def test_format_conversion_result_temperature_partial_match():
    # 'C' - температура, 'метр' - нет. Должно использовать символы температуры.
    result = uf.format_conversion_result(0, "c", 273.15, "метр")
    assert "0°C = 273.15K" not in result # Убеждаемся, что логика температур не
    смешивается с линейной

    # Этот тест проверяет, что блок IF выполняется, даже если одна из единиц имеет
    неизвестный символ

    # Мутационное тестирование показало, что блок IF должен быть чувствительным:

    # Добавим тест с неизвестным символом для убийства мутанта (any/all):
    result = uf.format_conversion_result(100, "цельсий", 273.15, "неизвестная_ед")
    # Должен сработать блок температур, поскольку "цельсий" есть в списке
    assert "°C" in result
    assert "неизвестная_ед" in result

    # Если бы было `all`, этот тест провалился бы. Следовательно, этот тест
    убивает мутанта.
```


3 АНАЛИЗ И ВЫВОДЫ

3.1 Оценка качества тестирования

Итоговое Качество Тестирования:

1. Полнота (Покрытие): Теперь тесты обеспечивают не только высокое покрытие строк кода, но и высокое покрытие логики. Почти все значимые операторы (математические, логические, сравнения) проверяются на их чувствительность к ошибкам.
2. Надежность (Чувствительность): Набор тестов способен обнаруживать даже атомарные, синтаксически небольшие ошибки (мутанты), которые могли бы быть допущены программистом, что является прямым доказательством его надежности.
3. Тестирование интеграционных точек: Особое внимание уделено интеграционным точкам (например, view вызывает validator и history). Проверка, что удаление валидации приводит к сбою, а удаление записи в историю обнаруживается, делает интеграционные тесты более надежными.

3.2 Анализ недостатков и их устранение

Улучшения, которые были достигнуты благодаря мутационному тестированию:

Модуль	Ранее Выжившие Мутанты (Пробелы)	Статус после изменений
unit_formatter	Пропуски в проверке пограничных случаев форматирования чисел (научная нотация, конечная точка).	Убиты. Добавлены тесты, которые: 1) явно проверяют удаление конечной точки (<code>5.0</code> $\rightarrow$ <code>5</code>); 2) проверяют научную нотацию для пограничных значений (<code>10⁶</code> , <code>10⁻⁴</code>).

3.3 Итоговые выводы по результатам работы

В результате выполнения практической работы были достигнуты поставленные цели: освоены методы модульного и мутационного тестирования, разработаны и проанализированы тестовые наборы. В ходе исследования успешно применено мутационное тестирование, созданы эффективные модульные тесты с высоким покрытием кода. Мутационное тестирование позволило выявить и устранить пробелы в тестовом наборе, повысив качество проверки программного продукта. Приобретены практические навыки разработки тестов, анализа покрытия кода и оценки эффективности тестовых сценариев. Полученные результаты подтверждают, что комбинированный подход к тестированию обеспечивает надёжную проверку качества программного обеспечения и помогает обнаруживать потенциальные ошибки на ранних этапах разработки.